

UNIVERSIDAD DEL VALLE DE GUATEMALA

Facultad de Ingeniería
Departamento de Ciencias de la Computación

Screensaver Paralelo con OpenMP

Proyecto No. 1

Curso: Computación Paralela y Distribuida
Sección: 30
Catedrático: Marlón Fuentes

Jose Ordoñez 231329
Pablo Méndez 23975
Osman De León 23428

Guatemala, septiembre de 2026

Repositorio: <https://github.com/JoseOrdonezB/screensaver-proyecto1.git>

Índice

1. Introducción	1
2. Antecedentes	1
3. Marco teórico	1
3.1. Computación paralela y memoria compartida	1
3.2. Distribución de iteraciones	1
3.3. Colisiones, speedup y eficiencia	2
4. Justificación y objetivos	2
4.1. Objetivo general	2
4.2. Objetivos específicos	2
5. Diseño y funcionamiento del programa	2
6. Implementación	3
7. Paralelización	3
7.1. Aplicación del método PCAM	3
8. Pruebas y resultados	3
8.1. Movimiento	4
8.2. Simulación completa	4
8.3. Comparación de schedules	5
9. Conclusiones	6
10.Recomendaciones	6
11.Referencias	6
A. Diagrama de flujo del programa	8
B. Catálogo de funciones	9
B.1. Simulación, movimiento y colisiones	9
B.2. Renderizado y ejecución	9
B.3. Benchmark y validación	10

C. Bitácora de pruebas	11
C.1. Validación funcional	11
C.2. Diez mediciones: movimiento	11
C.3. Diez mediciones: simulación completa	12
C.4. Comparación de schedules	12
C.5. Extracto de las mediciones	12

1. Introducción

Las computadoras actuales cuentan con varios núcleos de procesamiento, pero un programa secuencial no los aprovecha automáticamente. La computación paralela permite dividir parte del trabajo entre varios hilos y, cuando la carga lo justifica, reducir el tiempo de ejecución. OpenMP facilita este proceso mediante directivas y funciones para programación de memoria compartida [2].

Este proyecto desarrolla un screensaver de partículas en C++17 y SDL2. Las partículas se generan de forma pseudoaleatoria, se desplazan dentro del canvas, rebotan contra los bordes y pueden colisionar entre sí. Primero se construyó una versión secuencial funcional y posteriormente se identificaron dos partes con potencial de paralelización: el movimiento y la detección de colisiones.

El trabajo no consistió únicamente en agregar hilos. Las colisiones involucran memoria compartida y una estrategia de sincronización inadecuada puede eliminar cualquier mejora. Por esta razón se probaron distintas formas de distribuir el trabajo y se compararon las versiones mediante tiempos de ejecución, speedup y eficiencia.

2. Antecedentes

La evolución hacia procesadores multinúcleo ha hecho necesario diseñar programas capaces de aprovechar varios recursos de cómputo de forma simultánea. En un modelo de memoria compartida, varios hilos acceden al mismo espacio de memoria, por lo que es necesario distinguir entre operaciones independientes y operaciones que requieren coordinación.

OpenMP es una API para C, C++ y Fortran que permite incorporar paralelismo de forma incremental. Su modelo incluye regiones paralelas, distribución de iteraciones, sincronización y rutinas de control de ejecución [2]. Para organizar el diseño paralelo se tomó como referencia el método PCAM de Foster, basado en particionamiento, comunicación, aglomeración y mapeo [1].

3. Marco teórico

3.1. Computación paralela y memoria compartida

La computación paralela busca ejecutar varias unidades de trabajo de manera simultánea. OpenMP utiliza un modelo de memoria compartida en el que los hilos pueden consultar y modificar estructuras comunes. Esto simplifica el intercambio de datos, pero obliga a evitar carreras de datos y sincronización innecesaria.

3.2. Distribución de iteraciones

OpenMP permite controlar cómo se reparten las iteraciones de un ciclo. En este proyecto se compararon tres esquemas mediante `schedule(runtime)`:

- **static**: distribuye bloques de iteraciones antes de iniciar el trabajo.
- **dynamic**: entrega nuevos bloques conforme los hilos terminan los anteriores.
- **guided**: utiliza bloques decrecientes para combinar balanceo y menor overhead.

La especificación y la guía de referencia de OpenMP describen estas políticas y permiten seleccionarlas en tiempo de ejecución con `omp_set_schedule()` [2, 4].

3.3. Colisiones, speedup y eficiencia

Dos partículas se consideran en contacto cuando la distancia entre sus centros es menor que la suma de sus radios. La resolución corrige el solapamiento y aplica un impulso considerando masa, dirección relativa y restitución.

El speedup compara el tiempo secuencial con el paralelo:

$$S_p = \frac{T_s}{T_p}$$

La eficiencia relaciona el speedup con la cantidad de hilos p :

$$E_p = \frac{S_p}{p}$$

Un speedup mayor que 1 representa una reducción del tiempo respecto a la referencia secuencial.

4. Justificación y objetivos

El screensaver permite estudiar paralelismo sobre una carga visual y medible. El movimiento de cada partícula es independiente, mientras que las colisiones introducen dependencias y acceso a memoria compartida. Esta diferencia permite observar tanto casos sencillos de paralelización como problemas reales de sincronización.

4.1. Objetivo general

Aplicar el método PCAM para transformar un programa secuencial en una versión paralela utilizando OpenMP y evaluar la mejora de rendimiento obtenida.

4.2. Objetivos específicos

- Implementar un screensaver de partículas funcional y parametrizable.
- Identificar las partes del programa con potencial de paralelización.
- Desarrollar una versión paralela gestionando correctamente la memoria compartida.
- Medir tiempos de ejecución y calcular speedup y eficiencia.
- Comparar distintas cantidades de hilos y políticas de distribución.

5. Diseño y funcionamiento del programa

El programa se organizó en módulos para separar creación de partículas, movimiento, colisiones y renderizado. SDL2 se utiliza para crear la ventana, manejar eventos y presentar el framebuffer [3]. La simulación repite tres etapas principales: procesar eventos, actualizar el estado y renderizar el frame.

En la versión secuencial, el movimiento se actualiza partícula por partícula y las colisiones se revisan por parejas. En la versión paralela, OpenMP distribuye el movimiento y la detección de posibles colisiones. La configuración se recibe por argumentos de línea de comandos y se valida antes de iniciar la simulación.

6. Implementación

El programa fue desarrollado en C++17 utilizando SDL2 para el renderizado y OpenMP para la versión paralela. Cada partícula posee posición, velocidad, radio, masa y color. Durante cada frame se actualiza su posición, se verifican rebotes contra el canvas y se procesan colisiones. Cuando dos partículas se intersectan, se corrige el solapamiento y se aplica un impulso para modificar sus velocidades.

Se implementaron dos ejecutables principales: **screensaver**, correspondiente a la versión secuencial, y **screensaver_parallel**, que utiliza OpenMP. El programa permite configurar cantidad de partículas, dimensiones, semilla, límite de FPS y, en la versión paralela, cantidad de hilos. También se desarrollaron herramientas de validación y un benchmark que realiza diez repeticiones por configuración y exporta los resultados a CSV.

7. Paralelización

El movimiento se paralelizó con **parallel for**, ya que cada iteración modifica una partícula distinta. En este caso no es necesario utilizar **critical**, **atomic** o **mutex**.

Las colisiones requirieron una estrategia distinta. Una primera versión protegía cada pareja con una sección **critical**; aunque era segura, generaba demasiada espera. La solución final separó el proceso en dos etapas: primero se detectan posibles colisiones en paralelo y cada hilo guarda sus parejas en una lista local; después de la sincronización, las parejas encontradas se resuelven. Esto reduce la contención y evita escrituras concurrentes sobre las partículas durante la detección.

7.1. Aplicación del método PCAM

En el **particionamiento**, el trabajo se dividió por partículas y por parejas de partículas. En la **comunicación**, los hilos comparten el arreglo de partículas, utilizando solo lecturas durante la detección. En la **aglomeración**, cada hilo agrupa las colisiones detectadas en una lista local. Finalmente, en el **mapeo**, OpenMP distribuye las iteraciones mediante los esquemas **static**, **dynamic** y **guided**.

8. Pruebas y resultados

Antes de medir rendimiento se verificó el comportamiento de las colisiones con pruebas controladas y un stress test de 500 partículas y 20 repeticiones. La versión paralela se validó con 1, 2, 4 y 8 hilos. Para rendimiento, cada configuración del benchmark se ejecutó 10 veces y se registraron tiempo secuencial, tiempo paralelo, speedup y eficiencia.

8.1. Movimiento

La prueba de movimiento utilizó 1,000, 10,000 y 100,000 partículas durante 200 frames. La carga por partícula es similar, por lo que static presenta poco overhead. Conforme aumenta la cantidad de partículas, existe suficiente trabajo para compensar el costo de coordinación de los hilos.

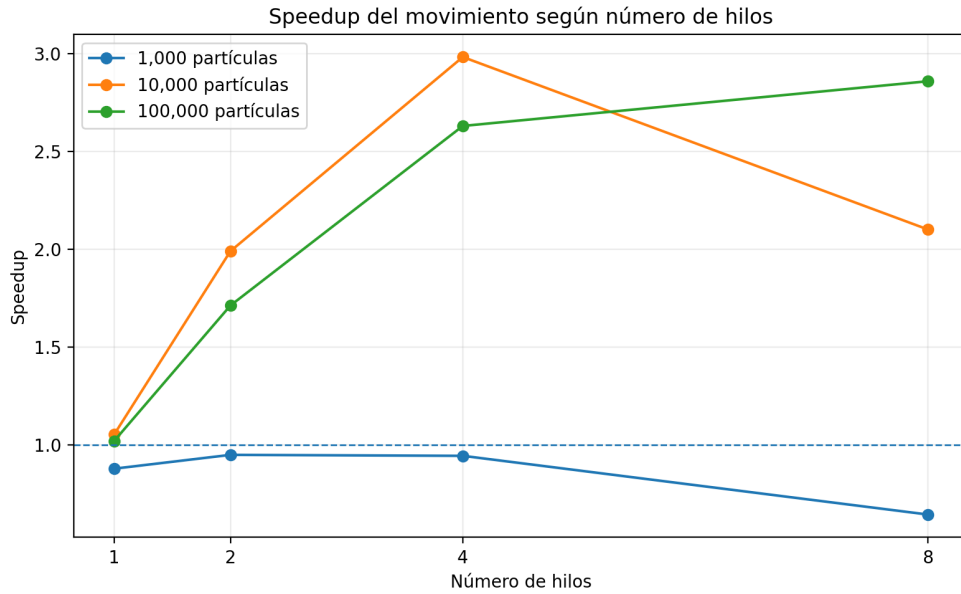


Figura 1: Speedup del movimiento según cantidad de hilos.

Con 1,000 partículas el overhead limita la mejora y puede hacer que la versión paralela sea más lenta. En cambio, con 10,000 y 100,000 partículas se observan speedups cercanos a 3. En la configuración de 10,000 partículas, 4 hilos y schedule static, el speedup promedio fue 2.98.

8.2. Simulación completa

La segunda prueba incluyó movimiento y colisiones. Se utilizaron 250, 500 y 1,000 partículas durante 50 frames. Se eligieron cantidades menores porque la búsqueda directa de parejas tiene costo aproximado $O(N^2)$.

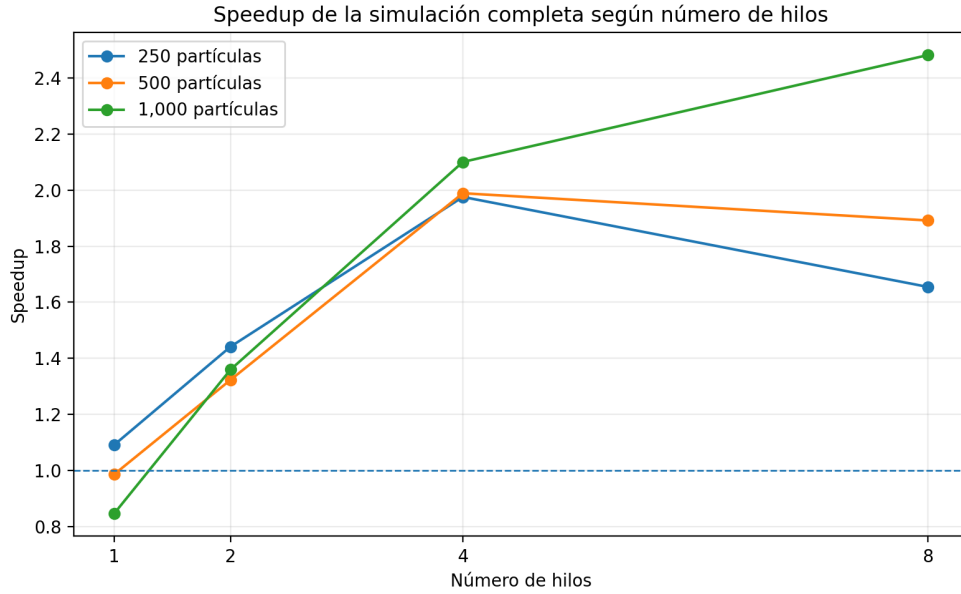


Figura 2: Speedup de la simulación completa según cantidad de hilos.

La versión paralela redujo el tiempo de ejecución en las cargas principales. El mayor speedup promedio del conjunto final fue 2.48 con 1,000 partículas, 8 hilos y schedule dynamic. La eficiencia disminuye al agregar hilos, lo que indica que el rendimiento no escala de forma proporcional.

8.3. Comparación de schedules

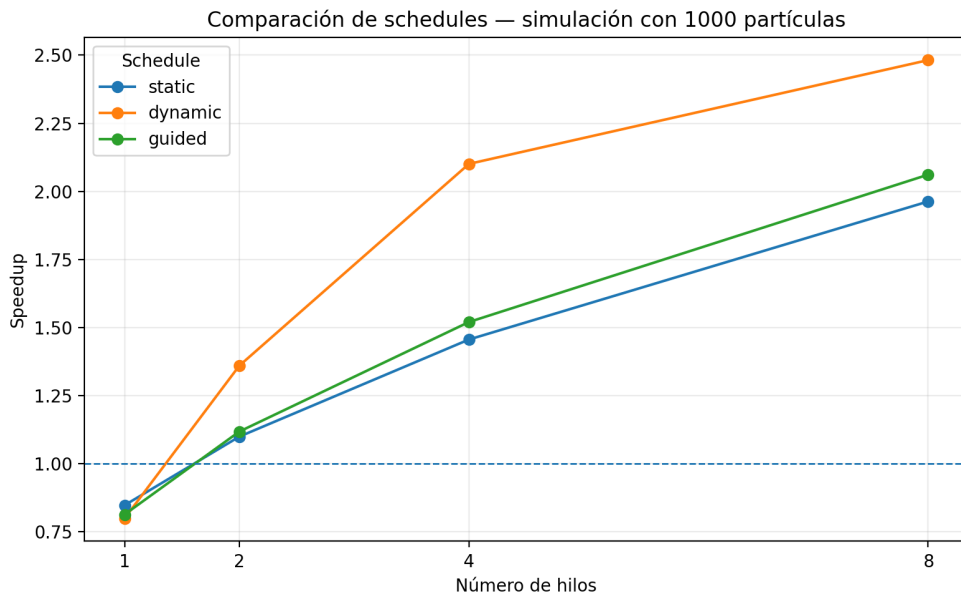


Figura 3: Comparación de schedules en la simulación con 1,000 partículas.

No existe una política que sea superior en todos los casos. Para movimiento uniforme, static evita parte del overhead de la asignación dinámica. En la simulación con 1,000 partículas, dynamic obtuvo el mejor resultado con 8 hilos, alcanzando un speedup promedio de 2.48. Esto confirma que la política debe seleccionarse a partir de mediciones y del tipo de carga.

Cuadro 1: Mejores configuraciones observadas en la simulación completa.

Partículas	Hilos	Schedule	Speedup	Eficiencia
250	4	static	1.976	49.4 %
500	4	guided	1.989	49.7 %
1000	8	dynamic	2.482	31.0 %

9. Conclusiones

El proyecto permitió comprobar que OpenMP puede mejorar el rendimiento de una simulación de partículas cuando existe suficiente trabajo para distribuir. Los mejores resultados no se obtuvieron necesariamente con la mayor cantidad de hilos, ya que el overhead y la sincronización también influyen en el tiempo de ejecución.

La paralelización del movimiento fue directa debido a la independencia entre partículas. Las colisiones requirieron modificar la estrategia inicial: proteger cada pareja con `critical` provocaba una pérdida importante de rendimiento, mientras que separar la detección paralela de la resolución redujo la contención.

Los resultados también mostraron que static, dynamic y guided responden de forma diferente según la carga. Las mediciones de speedup y eficiencia fueron necesarias para seleccionar configuraciones con base en evidencia y no únicamente en la cantidad de hilos disponible.

10. Recomendaciones

Como mejora futura se recomienda utilizar una estructura espacial, por ejemplo una cuadrícula uniforme, para reducir la cantidad de parejas revisadas durante la detección de colisiones. El enfoque actual compara pares de forma directa y su costo crece aproximadamente como $O(N^2)$.

También se recomienda realizar las mediciones finales con compilación optimizada y condiciones similares entre ejecuciones. Antes de seleccionar una cantidad de hilos o un schedule conviene probar la carga real, ya que la mejor configuración puede variar.

11. Referencias

Referencias

- [1] Foster, I. (1995). *Designing and Building Parallel Programs: Concepts and Tools for Parallel Software Engineering*. Addison-Wesley. Disponible en: <https://www.mcs.anl.gov/~itf/dbpp/>

- [2] OpenMP Architecture Review Board. (2024). *OpenMP Application Programming Interface, Version 6.0*. Disponible en: <https://www.openmp.org/specifications/>
- [3] Simple DirectMedia Layer. (s. f.). *SDL2 Documentation*. SDL Wiki. Disponible en: <https://wiki.libsdl.org/SDL2/FrontPage>
- [4] OpenMP Architecture Review Board. (2024). *OpenMP API 6.0 Reference Guide*. Disponible en: <https://www.openmp.org/resources/refguides/>

A. Diagrama de flujo del programa

El diagrama de la Figura 4 resume la ejecución de la versión paralela. La validación de argumentos representa la programación defensiva; el movimiento y la detección de colisiones corresponden a las secciones paralelas. Al finalizar la detección, la barrera implícita de OpenMP garantiza que las listas locales estén completas antes de resolver las parejas encontradas.

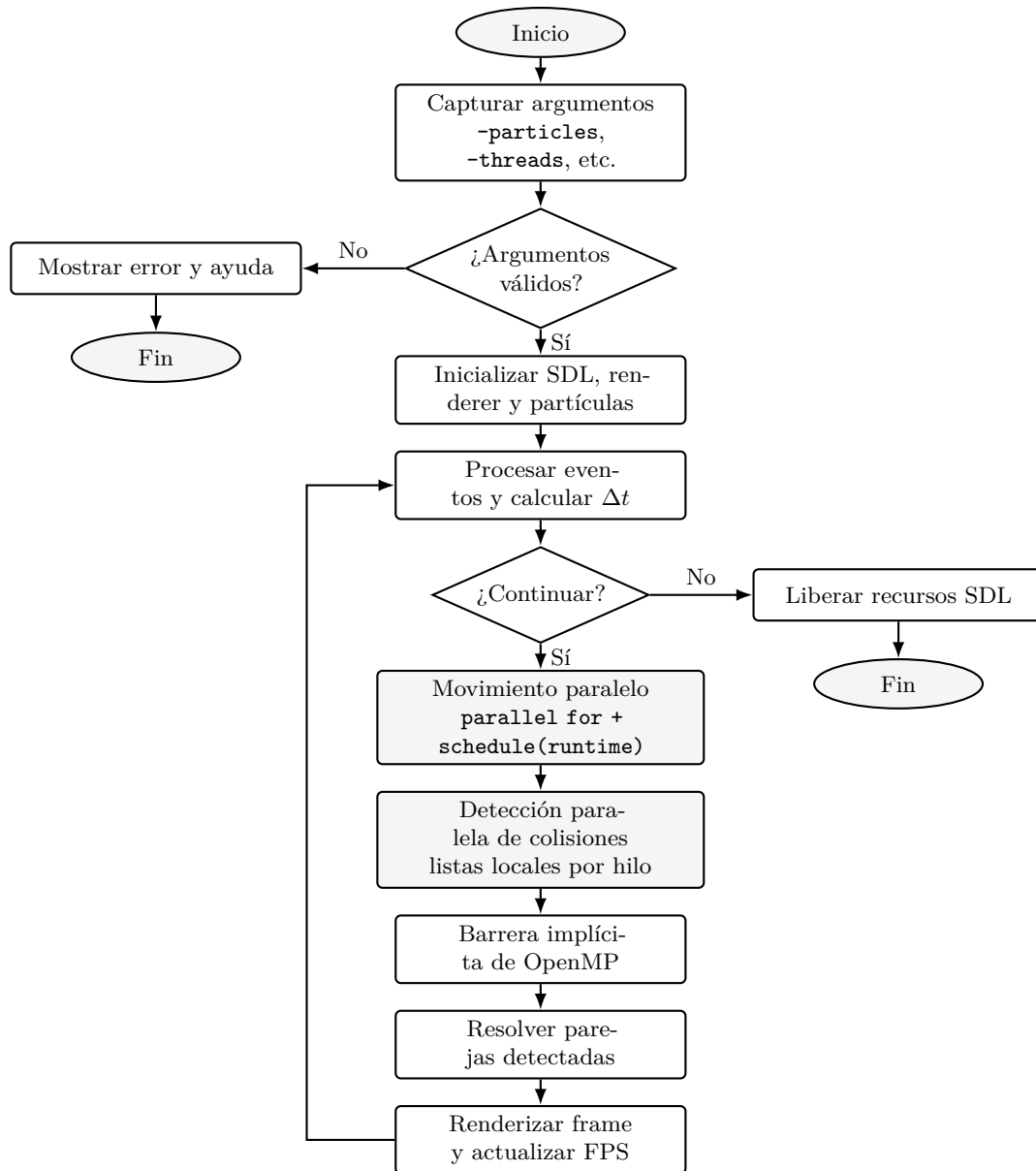


Figura 4: Flujo general de la versión paralela del screensaver. Fuente: elaboración propia.

B. Catálogo de funciones

El catálogo se presenta por módulo para facilitar su lectura. Se incluyen las funciones principales y las auxiliares que intervienen directamente en el flujo de la simulación, la paralelización, las pruebas y la medición de rendimiento.

B.1. Simulación, movimiento y colisiones

Cuadro 2: Funciones de simulación, movimiento y colisiones.

Función	Entradas	Salida	Descripción
<code>initializeParticles</code>	vector de partículas, configuración	void	Crea e inicializa las partículas con la configuración y semilla indicadas.
<code>updateSequential</code>	partículas, configuración	void	Actualiza movimiento y rebotes de todas las partículas de forma secuencial.
<code>updateParallel</code>	partículas, configuración	void	Distribuye la actualización de partículas con OpenMP y <code>schedule(runtime)</code> .
<code>resolveWallBounce</code>	partícula, configuración	void	Corrige posición y velocidad cuando una partícula alcanza un borde del canvas.
<code>applySchedule</code>	configuración	void	Configura <code>static</code> , <code>dynamic</code> o <code>guided</code> mediante <code>omp_set_schedule</code> .
<code>resolveCollisionsSequential</code>	partículas, configuración	void	Revisa las parejas y resuelve las colisiones de forma secuencial.
<code>resolveCollisionsParallel</code>	partículas, configuración	void	Detecta parejas en paralelo mediante listas locales y posteriormente resuelve las encontradas.
<code>particlesOverlap</code>	dos partículas	bool	Determina si la distancia entre centros es menor que la suma de sus radios.
<code>resolveParticlePair</code>	dos partículas, configuración	void	Corrige el solapamiento y aplica el impulso de colisión.
<code>normalizePair</code>	dx, dy	normal X/Y	Normaliza el vector entre dos partículas evitando una división por cero.

B.2. Renderizado y ejecución

Cuadro 3: Funciones de renderizado y ejecución.

Función	Entradas	Salida	Descripción
<code>initRenderer</code>	renderer, ancho, alto	bool	Crea la ventana, el renderer SDL, la textura y el framebuffer.
<code>clearRenderer</code>	renderer	void	Limpia el framebuffer con el color de fondo.
<code>drawRendererSequential</code>	renderer, partículas	void	Dibuja las partículas y sus halos en el framebuffer.
<code>presentRenderer</code>	renderer	void	Actualiza la textura y presenta el frame en pantalla.
<code>shutdownRenderer</code>	renderer	void	Libera textura, renderer, ventana y memoria asociada.
<code>parseArguments</code>	<code>argc</code> , <code>argv</code> , argumentos	bool	Lee los parámetros de línea de comandos y detecta errores de formato.
<code>validateArguments</code>	argumentos	bool	Verifica rangos, obligatoriedad y valores válidos antes de iniciar.

Función	Entradas	Salida	Descripción
<code>computeDeltaTime</code>	tiempo anterior	float	Calcula el tiempo transcurrido entre frames dentro de un rango seguro.
<code>runGameLoop</code>	renderer, partículas, configuración, FPS	void	Ejecuta eventos, simulación, renderizado y control de FPS.

B.3. Benchmark y validación

Cuadro 4: Funciones de benchmark y validación.

Función	Entradas	Salida	Descripción
<code>measureMovementSequential</code>	partículas iniciales, configuración, frames	double	Mide el tiempo del movimiento secuencial.
<code>measureMovementParallel</code>	partículas iniciales, configuración, frames	double	Mide el tiempo del movimiento paralelo.
<code>measureSimulationSequential</code>	partículas iniciales, configuración, frames	double	Mide movimiento y colisiones secuenciales.
<code>measureSimulationParallel</code>	partículas iniciales, configuración, frames	double	Mide movimiento y colisiones paralelas.
<code>writeMeasurement</code>	archivo, configuración, tiempos	void	Guarda una repetición con speedup y eficiencia en el CSV.
<code>validateSequentialCollision</code>	sin parámetros	bool	Ejecuta el escenario controlado con la versión secuencial.
<code>validateParallelCollision</code>	cantidad de hilos	bool	Ejecuta el escenario controlado con la versión paralela.
<code>runParallelStressTest</code>	hilos, repeticiones, partículas	bool	Repite la resolución paralela y comprueba que los estados numéricos sean válidos.

C. Bitácora de pruebas

El benchmark final contiene 720 mediciones individuales: 360 de movimiento y 360 de simulación completa. Cada combinación de cantidad de partículas, hilos y *schedule* se ejecutó 10 veces. El archivo completo `resultados_benchmark.csv` se conserva como material suplementario.

C.1. Validación funcional

Antes del benchmark se ejecutó `collision_validation`. La colisión secuencial, las versiones paralelas con 1, 2, 4 y 8 hilos, el rebote contra pared y el *stress test* de 500 partículas durante 20 repeticiones finalizaron correctamente.

Cuadro 5: Resumen de validaciones funcionales.

Prueba	Resultado
Colisión secuencial controlada	OK
Paralelo - 1 hilo	OK
Paralelo - 2 hilos	OK
Paralelo - 4 hilos	OK
Paralelo - 8 hilos	OK
Rebote contra pared	OK
Stress test: 500 partículas, 20 repeticiones	OK

C.2. Diez mediciones: movimiento

La Tabla 6 muestra las diez repeticiones de una configuración representativa: 10,000 partículas, 200 frames, 4 hilos y *schedule* static.

Cuadro 6: Diez mediciones de movimiento: 10,000 partículas, 4 hilos, static.

Rep.	Sec. (ms)	Par. (ms)	Speedup	Efic.
1	5.540	2.228	2.486	62.1 %
2	4.499	1.310	3.435	85.9 %
3	4.099	1.277	3.210	80.3 %
4	4.255	1.265	3.363	84.1 %
5	3.969	1.833	2.165	54.1 %
6	3.857	1.229	3.139	78.5 %
7	4.006	1.279	3.133	78.3 %
8	3.853	1.238	3.111	77.8 %
9	3.815	1.218	3.132	78.3 %
10	3.788	1.425	2.658	66.5 %
Promedio	4.168	1.430	2.983	74.6 %

C.3. Diez mediciones: simulación completa

La Tabla 7 muestra una configuración de carga alta: 1,000 partículas, 50 frames, 8 hilos y *schedule* dynamic.

Cuadro 7: Diez mediciones de simulación: 1,000 partículas, 8 hilos, dynamic.

Rep.	Sec. (ms)	Par. (ms)	Speedup	Efic.
1	27.104	6.822	3.973	49.7 %
2	19.627	8.618	2.278	28.5 %
3	18.359	7.766	2.364	29.6 %
4	18.721	7.396	2.531	31.6 %
5	18.066	7.622	2.370	29.6 %
6	18.015	7.967	2.261	28.3 %
7	18.268	7.525	2.428	30.3 %
8	18.156	8.410	2.159	27.0 %
9	18.062	8.141	2.219	27.7 %
10	18.165	8.124	2.236	27.9 %
Promedio	19.254	7.839	2.482	31.0 %

C.4. Comparación de schedules

Cuadro 8: Promedios para simulación con 1,000 partículas.

Schedule	Hilos	Par. (ms)	Speedup	Eficiencia
static	2	17.485	1.099	54.9 %
static	4	13.986	1.455	36.4 %
static	8	9.945	1.962	24.5 %
guided	2	17.229	1.117	55.9 %
guided	4	12.661	1.520	38.0 %
guided	8	9.485	2.061	25.8 %
dynamic	2	14.139	1.359	68.0 %
dynamic	4	9.239	2.101	52.5 %
dynamic	8	7.839	2.482	31.0 %

C.5. Extracto de las mediciones

Como referencia, una ejecución representativa de la simulación completa con 1,000 partículas y 50 frames produjo los siguientes promedios usando *schedule* dynamic:

2 hilos: speedup promedio 1.359

4 hilos: speedup promedio 2.101

8 hilos: speedup promedio 2.482

Los resultados completos, incluidas las diez repeticiones de cada configuración, se almacenaron en `resultados_benchmark.csv`. La variación entre repeticiones es esperable debido al estado del sistema operativo, cachés y planificación de hilos; por ello se reportan promedios y se conservan las mediciones individuales como evidencia.